Cloud Security Essentials TASK -2

1. **Log in to your AWS account and use AWS Key Management Service (KMS) to create a new symmetric customer managed key named 'FoodieAppKey' for encrypting Zomato-style order data. Note down the key ARN and set an alias for easy identification.**

   Ans :

   - Log in to the AWS Management Console.
   - In the search bar, search for Key Management Service (KMS) and open it.
   - From the left navigation pane, click Customer managed keys.
   - Click Create key.
   - Under Key type, select Symmetric.
   - Under Key usage, select Encrypt and decrypt.
   - Click Next.
   - Enter the key alias as:
   - FoodieAppKey
   - Click Next.
   - Select the IAM users or roles who will administer the key.
   - Click Next.
   - Select the IAM users or roles allowed to use the key for encryption and decryption.
   - Click Next.
   - Review the key configuration.

- Click Finish to create the key.
- After the key is created, open the key details page.

2. **In AWS KMS, enable automatic key rotation for the 'FoodieAppKey' you created. Take a screenshot of the rotation status showing it is enabled.**

   Ans :

   - Log in to the AWS Management Console.
   - Open the AWS Key Management Service (KMS).
   - From the left navigation menu, click Customer managed keys.
   - Select the key named:
   - FoodieAppKey
   - Open the Key rotation tab (or go to the Cryptographic configuration section depending on the AWS console layout).
   - Click Edit next to Automatic key rotation.
   - Enable the option:
   - Enable automatic key rotation
   - Confirm the rotation setting.
   - Click Save changes.

3. **Write a short note comparing how key rotation is handled in AWS KMS versus Azure Key Vault. Mention at least two similarities and two differences.**

   Ans :

Feature	AWS KMS	Azure Key Vault
Key Rotation	AWS KMS supports automatic rotation of customer managed keys. When enabled, AWS automatically creates new key material periodically while keeping the same key ID and ARN.	Azure Key Vault supports automatic key rotation by creating new versions of keys according to a defined rotation policy.
Key Versions	Previous key versions are retained and can still be used to decrypt data encrypted with older key material.	Each rotation creates a new key version, while older versions remain available for decryption.
Key Management	Keys are managed through AWS KMS, and rotation settings are configured per customer managed key.	Keys are managed through Azure Key Vault, and rotation policies define when and how keys are rotated.
Automation	Rotation can be enabled automatically through the AWS Console, CLI, or SDK.	Rotation can be automated using Azure Key Vault rotation policies and Azure services.

4. **Use AWS KMS to generate a data encryption key (DEK) with the 'FoodieAppKey', then encrypt a sample order string (e.g.,**

**'OrderID: 12345, Item: Pizza') using the DEK. Show both the encrypted and decrypted outputs.

Hint: You can use the AWS CLI or AWS SDK for this task.**

Ans :

- Generate a Data Encryption Key (DEK) using the KMS key FoodieAppKey:
- aws kms generate-data-key \
- --key-id alias/FoodieAppKey \
- --key-spec AES_256
- The command returns two keys:
- Plaintext DEK → Used for encryption.
- Ciphertext DEK → Encrypted version stored safely using AWS KMS.
- Copy the Plaintext DEK value from the output and use it to encrypt the sample order data.
- Create a sample order text file:
- echo "OrderID: 12345, Item: Pizza" > order.txt
- Encrypt the order file using the generated DEK:
- openssl enc -aes-256-cbc \
- -salt \
- -pbkdf2 \
- -in order.txt \
- -out encrypted_order.txt \
- -pass pass:<PLAINTEXT_DEK>

5. **Use ChatGPT to draft a sample key rotation policy for a Flipkart-style e-commerce app, specifying rotation frequency, notification process, and steps to handle compromised keys. Paste the generated policy and highlight any changes you would make for your own app.**

   Ans :

   1. Purpose

   This policy defines the process for managing, rotating, and protecting encryption keys used by the Flipkart-style e-commerce application. The goal is to reduce security risks and protect sensitive data such as customer information, payment details, and order records.

   ---

   2. Key Rotation Frequency

   - Encryption keys must be rotated every **90 days** for critical systems such as payment processing and customer data storage.
   - Less sensitive keys may be rotated every **365 days**.
   - Automatic key rotation should be enabled wherever supported (for example, AWS KMS customer managed keys).
   - Emergency key rotation must be performed immediately if a key compromise is suspected.

   ---

   3. Key Rotation Process

   1. The security team schedules regular key rotation activities.
   2. A new encryption key is generated using the cloud key management service.
   3. The new key is tested before being used in production.

4. New data is encrypted using the new key.
5. Old keys are securely archived and retained only for decrypting previously encrypted data.
6. Access permissions are reviewed after every rotation.

---

4. Notification Process

- The security team receives alerts **30 days before scheduled key rotation**.
- Application owners are notified before production key changes.
- Successful key rotation events are logged and reported.
- Any failed rotation attempt generates an immediate security alert.
- All key management activities are monitored through audit logs.

---

5. Steps for Handling Compromised Keys

If a key compromise is detected:

1. Immediately disable the compromised key to prevent further use.
2. Notify the security team and system administrators.
3. Generate a new replacement encryption key.
4. Update applications and services to use the new key.
5. Re-encrypt sensitive data if required.
6. Review access logs to identify unauthorized usage.
7. Remove unnecessary permissions from affected users or services.

8. Document the incident and perform a security review.